MSc in Computer Science and Engineering

Planning, Learning and Decision Making

Lab 1: Markov Chains

Smart Home Energy Management

1 System Specification

You are tasked with modeling a Smart Home Energy System as a discrete-state Markov Chain. The system must be implemented according to the following factorization to ensure consistency across all student submissions. The questions might accept more than one correct solution, justify your options.

The state is defined by the tuple (B, W, C), where:

- Battery Level (B): $\{0, 1, 2, 3, 4\}$ (5 levels).
- Weather Situation (W): $\{0, 1, 2, 3, 4\}$ (5 levels: 0=Stormy, 4=Sunny). •

Consumption Level (C): $\{0, 1, 2\}$ (3 levels: 0=Low, 2=High).

Total operational states: $5 \times 5 \times 3 = 75$.
State 75 (Failure): A terminal "sink" state representing a permanent system failure.

1.1 Index Mapping

To represent the transition matrix $P \in \mathbb{R}^{76\times76}$, use the following lexicographical

indexing: $Index = (B \times 15) + (W \times 3) + C$

The failure state is assigned $Index = 75$.

Transitions occur in two phases: the physical update and the environmental

shift. 1. Battery Update: The next potential battery state B' is calculated as: $B' =$

$$\text{clip}(B + W - C, 0, 4)$$

2. Environmental Markov Chains:

- Weather ($W \rightarrow W_{next}$): Prob. 0.6 to stay, 0.2 to move to $W \pm 1$ (bounded by [0,4]). •

Consumption ($C \rightarrow C_{next}$): Prob. 0.7 to stay, 0.15 to move to $C \pm 1$ (bounded by [0,2]).

1.2 Scenarios

You must implement three versions of the transition matrix P:

- Scenario A (Robust): No transitions to State 75.
- Scenario B (Brittle-Bottom): There is a small probability of battery failure. We can implement this as follows: If the transition probability to state 0 is above 10%, remove exactly 10% from this probability and add it to the transition probability of state 75.
- Scenario C (Extreme-Stress): The battery is getting old and the probability of failure increases either when full or when empty. We can implement this as follows: For every next state where the probability is above 5% and either $B' = 0$ or $B' = 4$, remove exactly 5% from this probability and add it to the transition probability of state 75.

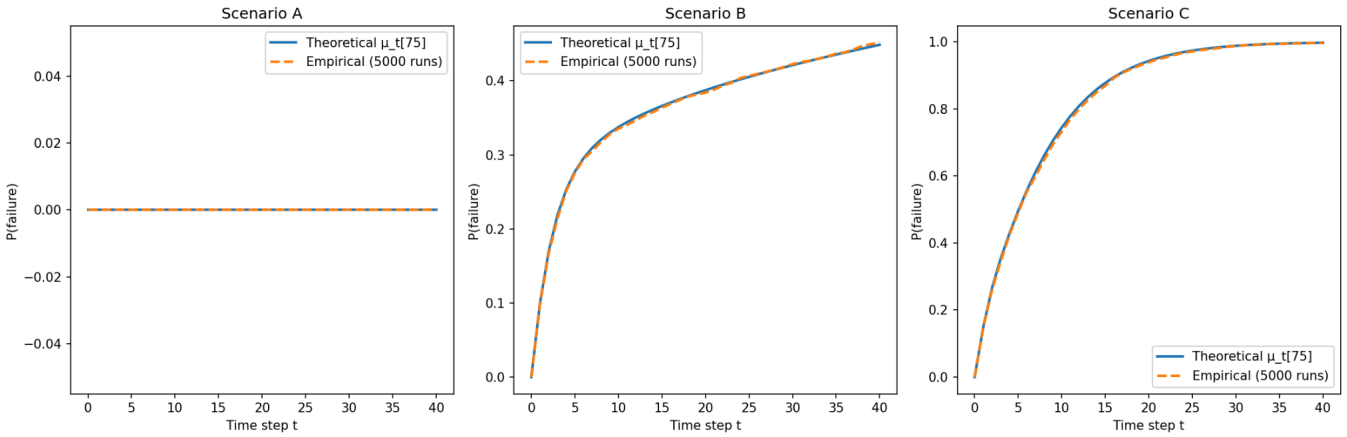
2 Questions (answers to be submitted)

1. What is a basic method to test if the transition matrix is wrong?
   Suggestion: Implement a function build_matrix(scenario) that returns the 76 × 76 tran sition matrix. Where scenario is either 0 (Scenario A), 1 (Scenario B), or 2 (Scenario C).

   R:. A basic method to test if the transition matrix is wrong is to check its shape, make sure that in every row, its sum is equal to 1 and that there are no negative probabilities.

2. Compare the theoretical probability of failure $\mu_t[75]$ (calculated via P^t) with the empirical fraction of failed simulations at each time step $t \in [0, 40]$. Explain how you did it and plot the results.
   Suggestion: Write a function that generates Nr trajectories of Ns steps starting in an initial state s_0.

   R:. For the theoretical probability of failure, we start with a 100% probability in s0, multiplying by P at each step to get to the next distribution and recording the probability of being in a failure state at each step. For the empirical failure, at each t € [0,40], we count the fraction of trajectories that are in state 75. We also simulate the trajectories by creating a Nr * Ns + 1 matrix of states, that in each time step, for each trajectory, samples the next state using the current row of P as probabilities.



As we can see, Scenario A stays at 0.00 due to not having transitions to State 75.

Scenario B shows that, with time, the probability of failure increases quickly and then slows down as the time steps advances.

Scenario C very quickly approaches P(failure) = 1.0 (around t=30), as expected due to its nature.

As expected, the theoretical and empirical comparisons are very similar.

3. How can you compute the stationary distribution of a Markov chain? All the methods give the same results? In which conditions?

   Suggestion: Given an initial state $s_0 = (B : 2, W : 2, C : 1)$, create two functions. One that computes the distribution μ_N after N steps, for instance using Power Iteration. One that computes the stationary distribution if it exists using eigen-decomposition.

R: The stationary distribution π is a probability distribution satisfying π P = π (it is a fixed point of the chain). There are two main computational methods:

1. Power Iteration:

   Starting from an initial distribution μ_0 (e.g., all mass on a single state), repeatedly multiply by P: $\mu_n = \mu_0 \cdot P^n$

   As N → ∞, μ_n may converge to the stationary distribution π. This is an approximation, you get closer with more iterations but never get the exact answer.

2. Eigen-decomposition:

   The equation π P = π means π is a left eigenvector of P with eigenvalue 1 (equivalently, a right eigenvector of P^T with eigenvalue 1). We solve this directly using linear algebra, then normalize so the entries sum to 1. This gives the exact stationary distribution (up to numerical precision).

From our experiments, they both give the same results as the maximum difference between the two was on the order of 10^{-15} (machine precision). But this is not always guaranteed as both methods only agree when the Markov chain is ergodic, meaning it satisfies two properties. These properties are that the chain is irreducible meaning that

every state can be reached from every other state and that it is aperiodic meaning that the chain does not cycle deterministically through states with a fixed period.

Under ergodicity, the stationary distribution is unique and power iteration converges to it regardless of the initial state. When these conditions are not met, the methods may diverge, for example if the chain is periodic, power iteration oscillates instead of converging, while eigen-decomposition still finds π. If the chain is reducible (multiple communicating classes), the stationary distribution is not unique eigen-decomposition returns one valid solution, while power iteration converges to a different one depending on the initial state s_0.

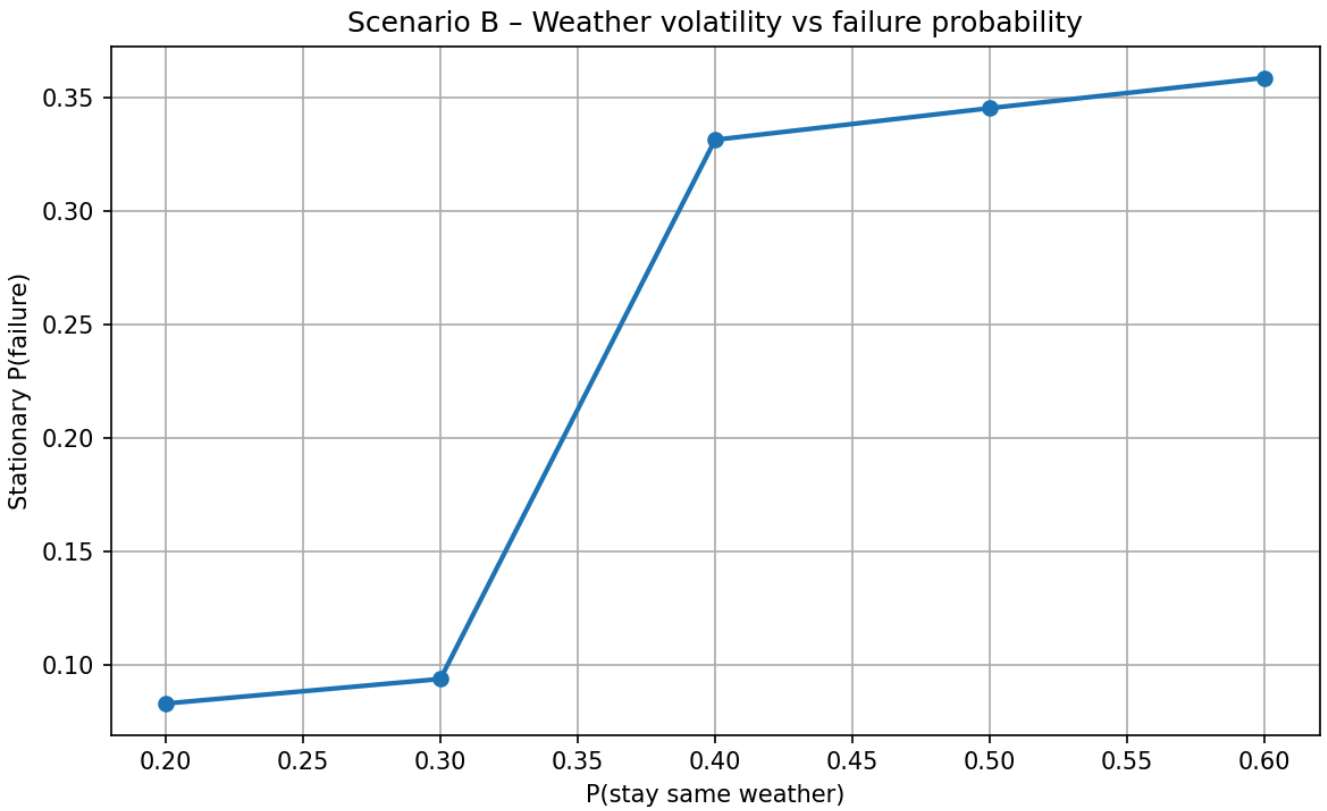
4. In Scenario C, the stationary distribution is mathematically trivial (100% failure). Explain why "Expected Time to Failure" is a more useful metric here.

   Suggestion: Compute the expect time to transition between states.

   R: In Scenario C, the stationary distribution is π = (0, 0, ..., 0, 1), meaning all probability mass on the absorbing failure state (state 75). This is mathematically trivial and tells us nothing beyond "the system will eventually fail with certainty." Every absorbing Markov chain has this same stationary distribution, regardless of how the system actually behaves before failure. Expected Time to Failure is far more informative because it answers the practical question: how long does the system survive before failing? Specifically, it tells us the expected number of steps before reaching the absorbing state, starting from any given state.

5. How does increasing the volatility of the Weather Markov Chain (e.g., lower probability of staying in the same state) affect the probability of failure in Scenario B?

   Suggestion: Try different values in the transition matrix and evaluate the stationary distribution.



R:. As seen by the plot, increasing the volatility of the Weather Markov Chain results in the weather changing frequently, so the battery gets mixed charging, resulting in a reduced probability of failure.

6. Based on your results, if a homeowner wants to avoid State 75 in Scenario C, should they aim for a battery level of 2 or 4?

   Suggestion: Justify using your transition matrix.
   R:. In scenario C, the probability of failure increases if the battery is full (B=4), and a battery level of 2 makes it less likely that the transactions will land in either of the levels that increase failure probability, making it the best option (This is backed up by the code done for the question, where after one step, B=2 has a 0% probability of failure and B=4 has a 25% to failure).

3 How to submit

You should submit one file called *padiproj1-groupXX.zip* with the 3 files:

1. *padihw1-groupXX.pdf* with the homework
2. *padilab1-groupXX.pdf* with the answers to the questions in this lab
3. *padilab1-groupXX.py* with the code developed

No other formats will be accepted. The homework and the answers to the lab questions can be either a photo of handwritten notes or in a digital format, just ensure that it is readable.